OmniFlux : Assets Aggregator

Fragmentasi alat keuangan menyulitkan pengguna melacak arus kas harian sekaligus memantau pertumbuhan aset investasi. Selain itu, agregator yang ada saat ini menimbulkan risiko privasi karena menyimpan jumlah kekayaan pengguna secara transparan.Sehingga diperlukan platform *WealthTech* yang mengotomatisasi pencatatan pengeluaran menggunakan kecerdasan buatan (AI) serta menyajikan dasbor agregator portofolio lintas instrumen (Kas, Saham, Kripto, Reksa Dana) dengan tingkat privasi tinggi.

1. Tinjauan Arsitektur Sistem

Bagian ini menyediakan representasi tingkat tinggi mengenai topologi sistem, menguraikan pola arsitektural yang dipilih. Deskripsi arsitektur memaparkan secara rasional bagaimana komponen-komponen berskala besar berinteraksi dan peran yang diberikan kepada setiap subsistem.

- **Frontend (NextJS):** Mengelola presentasi visual dinamis di sisi peramban klien.
- **Mobile (React Native) :** Sebagai pembuatan aplikasi mobile multi platform
- **Core Backend (Rust):** Bertindak sebagai *API Gateway* dan koordinator lalu lintas data.
- **Google Gemini API:** Pemrosesan *Natural Language Processing* (NLP).

2. Desain Data

Desain data merepresentasikan tulang punggung status sistem, berfokus pada manajemen, retensi, manipulasi, dan persistensi informasi. SDD mendefinisikan skema basis data, tata letak tabel (untuk basis data relasional), dan struktur relasional yang menghubungkan entitas data.

A. Tabel Users (Entitas Pengguna)

Pusat dari seluruh relasi data. Menyimpan mandat otentikasi dan kunci kriptografi.

Nama Kolom	Tipe Data MySQL	Keterangan & Konstrain
id	VARCHAR(36)	**Primary Key**. Menggunakan UUID agar sulit ditebak (meningkatkan keamanan).
email	VARCHAR(255)	**Unique**, NOT NULL. Alamat surel untuk otentikasi.

password_hash	VARCHAR(255)	NOT NULL. Kata sandi yang sudah di-hash (misal menggunakan Argon2).
created_at	TIMESTAMP	Waktu registrasi akun (DEFAULT CURRENT_TIMESTAMP).

B. Tabel Wallets (Sumber Dana / Kas)

Menyimpan daftar tempat penyimpanan uang tunai pengguna.

Nama Kolom	**Tipe Data MySQL**	**Keterangan & Konstrain**
id	INT	**Primary Key**, AUTO_INCREMENT.
user_id	VARCHAR(36)	**Foreign Key** ke Users(id). *Cascade Delete*.
name	VARCHAR(100)	NOT NULL. Nama dompet (Contoh: "BCA Utama", "Gopay", "Tunai").
balance	DECIMAL(15,2)	NOT NULL, Default 0.00. Saldo kas saat ini (disimpan *plain text* agar mudah diagregasi).
created_at	TIMESTAMP	Waktu pembuatan dompet.

C. Tabel Categories (Master Kategori AI)

Data referensi statis untuk pelabelan transaksi oleh Gemini API.

Nama Kolom	**Tipe Data MySQL**	**Keterangan & Konstrain**
id	INT	**Primary Key**, AUTO_INCREMENT.
name	VARCHAR(50)	NOT NULL. Nama kategori (Contoh: "Makanan", "Transportasi").
type	ENUM	NOT NULL. Nilainya hanya boleh: 'income' atau 'expense'.

D. Tabel Transactions (Arus Kas Harian)

Mencatat detail pemasukan dan pengeluaran.

Nama Kolom	Tipe Data MySQL	Keterangan & Konstrain
id	VARCHAR(36)	**Primary Key** (UUID).
user_id	VARCHAR(36)	**Foreign Key** ke Users(id). *Cascade Delete*.
wallet_id	INT	**Foreign Key** ke Wallets(id).
category_id	INT	**Foreign Key** ke Categories(id). Diisi otomatis oleh AI atau manual oleh pengguna.
amount	DECIMAL(15,2)	NOT NULL. Nilai transaksi.
description	TEXT	NOT NULL. Teks mentah yang di-*parsing* oleh AI (Contoh: "Beli bensin 50rb").
ai_confidence	DECIMAL(5,2)	Menyimpan persentase keyakinan AI (Contoh: 95.50). Jika NULL, berarti diinput manual.
transaction_date	TIMESTAMP	Waktu terjadinya transaksi.

E. Tabel Assets (Master Data Instrumen)

Data referensi publik untuk aset yang didukung platform.

Nama Kolom	Tipe Data MySQL	Keterangan & Konstrain
id	INT	**Primary Key**, AUTO_INCREMENT.
ticker_symbol	VARCHAR(20)	**Unique**, NOT NULL. (Contoh: "BBCA", "ETH"). Parameter untuk API eksternal.
asset_type	ENUM	NOT NULL. Nilai: 'Stock', 'Crypto', 'Mutual Fund'.
name	VARCHAR(100)	NOT NULL. Nama lengkap aset (Contoh: "Bank Central Asia Tbk").

F. Tabel Investment_Transactions (Histori Jual-Beli Aset)

Mencatat riwayat mutasi investasi pengguna.

Nama Kolom	Tipe Data MySQL	Keterangan & Konstrain
id	VARCHAR(36)	**Primary Key** (UUID).
user_id	VARCHAR(36)	**Foreign Key** ke Users(id). *Cascade Delete*.
asset_id	INT	**Foreign Key** ke Assets(id).
transaction_type	ENUM	NOT NULL. Nilai: 'buy' atau 'sell'.
transaction_date	TIMESTAMP	Waktu eksekusi investasi.

G. Tabel Portfolios (Agregasi Aset Terkini)

Menyimpan rangkuman status kepemilikan aset pengguna saat ini untuk mempercepat *load dashboard*.

Nama Kolom	Tipe Data MySQL	Keterangan & Konstrain
id	INT	**Primary Key**, AUTO_INCREMENT.
user_id	VARCHAR(36)	**Foreign Key** ke Users(id). *Cascade Delete*.
asset_id	INT	**Foreign Key** ke Assets(id).
updated_at	TIMESTAMP	Diperbarui setiap kali ada transaksi investasi baru.

H. Tabel Insights (Saran Perilaku AI)

Menyimpan notifikasi proaktif (*nudges*) yang di-*generate* oleh analitik perilaku.

Nama Kolom	Tipe Data MySQL	Keterangan & Konstrain
id	INT	**Primary Key**, AUTO_INCREMENT.
user_id	VARCHAR(36)	**Foreign Key** ke Users(id). *Cascade Delete*.

insight_type	VARCHAR(50)	Jenis saran (Contoh: 'budget_alert', 'investment_opportunity').
message	TEXT	NOT NULL. Pesan AI (Contoh: "Anda hemat 1 juta bulan ini, ETH sedang koreksi, ingin menambah muatan?").
is_read	BOOLEAN	Default FALSE. Menandai apakah saran sudah dibaca di *frontend* React.
created_at	TIMESTAMP	Waktu analitik di-*generate*.

3. Desain Antarmuka Pengguna

A. Warna Latar & Permukaan (Background & Surface)

Hindari penggunaan warna hitam pekat murni. Sebagai gantinya, gunakan gradasi warna gelap yang berlapis untuk menciptakan kedalaman spasial.

- **Latar Utama:** Gunakan abu-abu sangat gelap (hitam arang/obsidian). Ini berfungsi sebagai kanvas kosong yang tidak membuat mata lelah saat pengguna menganalisis grafik lama-lama.
- **Permukaan Kartu (Cards/Panels):** Untuk elemen seperti daftar transaksi atau kartu portofolio, gunakan warna abu-abu yang satu atau dua tingkat lebih terang dari latar utama. Perbedaan tipis ini menciptakan ilusi elevasi (seolah-olah kartu tersebut mengambang di atas latar belakang).

B. Warna Aksen Primer (Brand & Action)

Warna ini adalah identitas aplikasi Anda dan penunjuk arah bagi mata pengguna.

- **Biru Elektrik atau Ungu Neon:** Warna biru memicu perasaan aman dan terpercaya di dunia finansial konvensional, sementara tingkat kecerahan neon menyuntikkan nuansa futuristik dari sistem AI dan *blockchain* yang Anda bangun.
- **Penggunaan:** Aplikasikan warna ini secara hemat namun strategis—hanya pada tombol aksi utama (seperti tombol "Simpan"), ikon menu yang sedang aktif di navigasi bawah, atau garis kurva pada grafik *Net Worth* Anda.

C. Warna Semantik (Status Finansial)

Dalam UI/UX keuangan, warna semantik berbicara lebih keras daripada teks. Pengguna harus bisa mengetahui kondisi portofolionya dalam waktu kurang dari satu detik hanya dari lirikan warna.

- **Keuntungan & Pemasukan (Profit/Income):** Gunakan **Hijau Zamrud** (*Emerald Green*). Aplikasikan pada persentase PnL yang positif dan nominal uang masuk.
- **Kerugian & Pengeluaran (Loss/Expense):** Gunakan **Merah Kirmizi** (*Crimson Red*). Aplikasikan pada indikator koreksi pasar saham atau riwayat pembayaran tagihan.
- **Peringatan AI (AI Nudges):** Gunakan **Kuning Keemasan** atau **Jingga**. Warna ini memiliki kontras peringatan yang sangat kuat. Terapkan pada lencana (*badge*) saat AI Anda ragu mengategorikan transaksi, atau pada ikon lonceng saat ada pesan proaktif bahwa anggaran gaya hidup sudah menipis.

D. Warna Teks (Tipografi)

Hierarki informasi sangat krusial saat menampilkan deretan angka yang padat.

- **Teks Primer:** Gunakan putih bersih (atau *off-white*) untuk nominal saldo utama, *Net Worth*, dan judul halaman agar keterbacaannya maksimal.
- **Teks Sekunder:** Gunakan abu-abu terang redup untuk elemen yang kurang penting seperti stempel waktu (*timestamp*), label grafik bawah, atau nama bursa, agar tidak saling berebut perhatian dengan angka saldo utama.

E. Wireframe

Bagian ini menyajikan representasi visual (*wireframe*) dari tata letak antarmuka aplikasi OmniFlux. Rangkaian purwarupa ini dirancang khusus untuk memenuhi ruang lingkup *Minimum Viable Product* (MVP), dengan fokus memvisualisasikan fungsionalitas inti sistem. Desain struktural ini mengutamakan hierarki informasi yang jelas, kemudahan navigasi antar-modul, serta meminimalisasi elemen visual redundan untuk mempercepat pemuatan data. Melalui pemetaan visual pada layar Beranda, Transaksi, Portofolio, Notifikasi, dan Pengaturan berikut, tim pengembang memiliki acuan cetak biru (*blueprint*) yang presisi terkait alur pengalaman pengguna (UX) dan peletakan komponen antarmuka (UI) sebelum memasuki fase implementasi kode.

- Halaman Dashboard

[Label: Sapaan]
Halo, User

Total Kekayaan Bersih

Rp 150.000.000

[Sensor Privasi ON/OFF]

[Toggle]

Tren 30 Hari Terakhir

[PLACEHOLDER]
Line Chart Mini

Grafik performa kekayaan

3 Transaksi Terakhir

[Nama Transaksi 1] [Kategori]	**- Rp XXX.XXX**
[Nama Transaksi 2] [Kategori]	**- Rp XXX.XXX**
[Nama Transaksi 3] [Kategori]	**- Rp XXX.XXX**

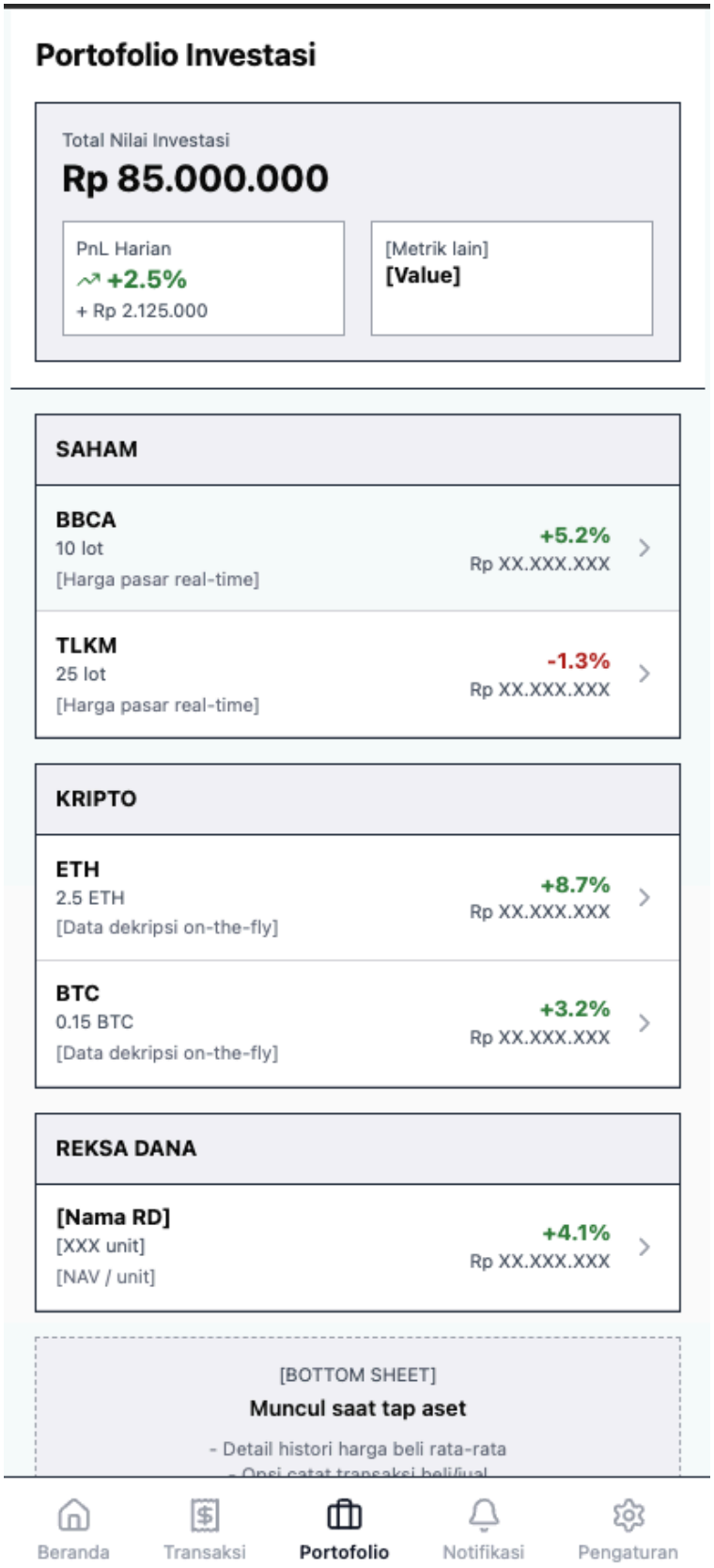
Highlight Portofolio Terbaik

ETH [Ethereum]	**+5.2%** [Hari ini]

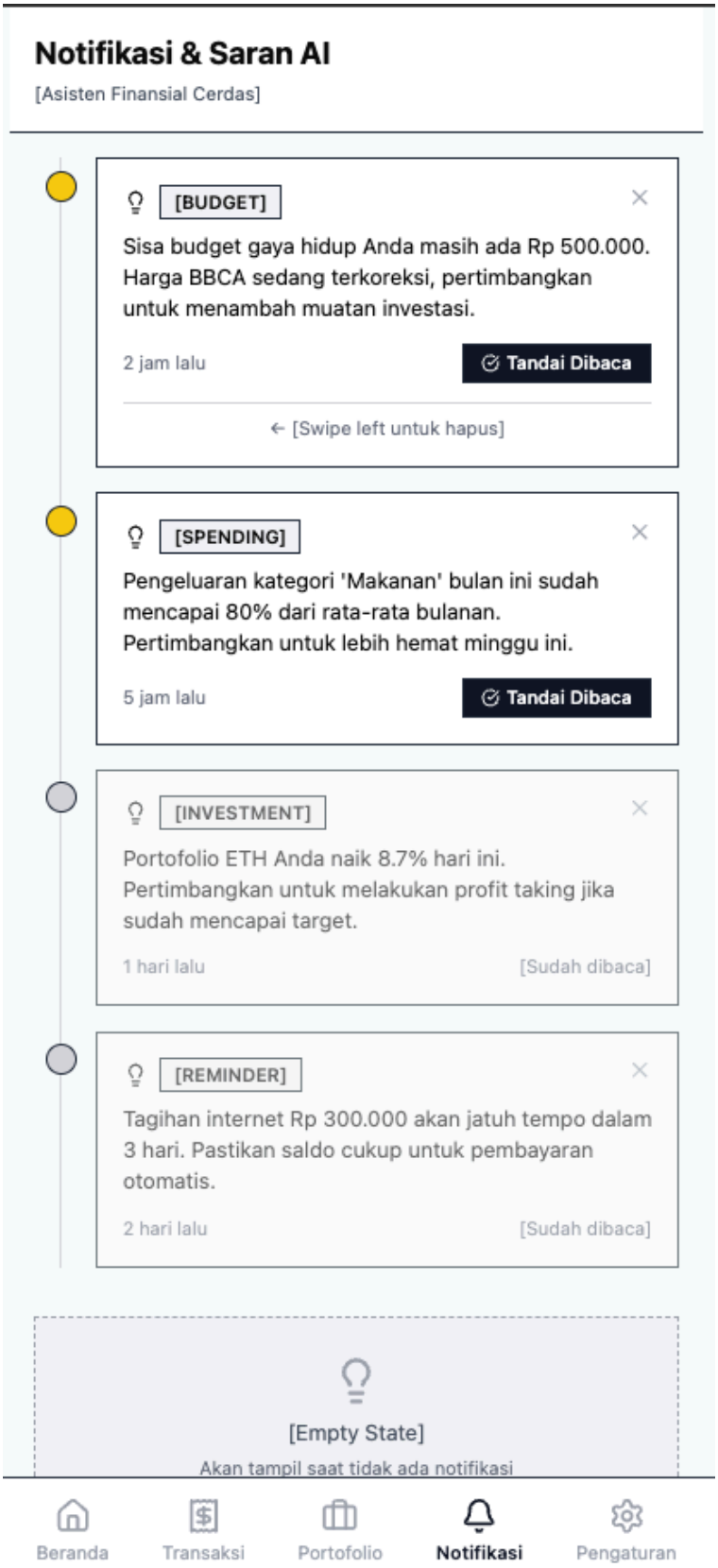
Beranda | Transaksi | Portofolio | Notifikasi | Pengaturan

- Halaman Transaksi

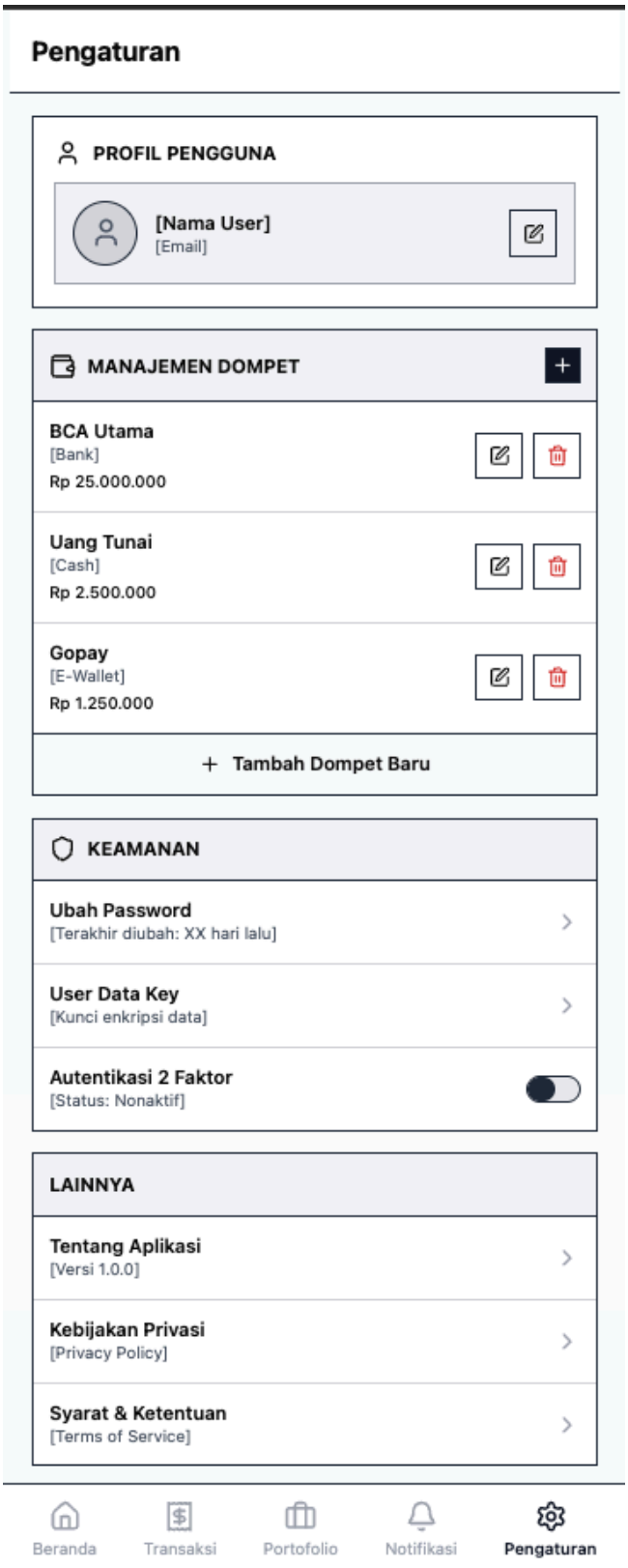
- Halaman Portofolio



- Halaman Notifikasi



- Halaman Pengaturan



F. Peta Alur Interaksi Pengguna

- **Alur Autentikasi & Inisialisasi (Gerbang Masuk)** Alur ini memastikan keamanan sebelum sesi aplikasi dimulai.
  1. **Layar Masuk (Login):** Pengguna membuka aplikasi dan dihadapkan pada form autentikasi.
  2. **Aksi - Kirim Mandat:** Pengguna memasukkan surel dan kata sandi, lalu menekan "Masuk".
  3. **Proses Latar Belakang:** Sistem memvalidasi ke Rust, menerima token JWT, dan memuat *User Data Key* ke dalam memori peramban/klien.
  4. **Hasil (Destinasi):** Navigasi diarahkan secara otomatis ke **Layar Beranda**.
- **Alur Pencatatan Arus Kas (Input Cerdas berbasis AI)** Alur ini adalah siklus interaksi harian utama pengguna.
  1. **Layar Transaksi:** Pengguna menavigasi ke tab "Transaksi" melalui *Bottom Navigation Bar*.
  2. **Aksi - Input Teks:** Pengguna mengetik kalimat alami di *Smart Input Field* (misal: "Bayar WiFi 300rb") dan menekan ikon "Kirim/Enter".
  3. **Status Transisi:** Layar memunculkan *Overlay/Spinner* memblokir interaksi sementara sistem menunggu balasan kategorisasi dari Gemini API
  4. **Hasil (Sukses):** Transaksi baru muncul di baris teratas "Riwayat Transaksi" dengan lencana kategori (misal: "Tagihan"). *Donut Chart* di tengah layar secara otomatis menggambar ulang proporsi pengeluaran.
  5. **Alur Alternatif (Koreksi AI):** Jika lencana kategori menampilkan ikon peringatan (!), pengguna mengetuk lencana tersebut -> Sistem memunculkan **Modal Daftar Kategori** -> Pengguna memilih kategori yang benar -> Sistem menyimpan revisi ke basis data.
- **Alur Agregasi dan Transaksi Investasi** Alur ini mengelola manipulasi data sensitif dengan enkripsi *on-the-fly*.
  1. **Layar Portofolio:** Pengguna menavigasi ke tab "Portofolio". Layar ini meminta data terenkripsi dari peladen dan merender *Net Worth* serta daftar aset secara instan.
  2. **Aksi - Lihat Detail Aset:** Pengguna mengetuk salah satu kartu aset (contoh: "ETH").
  3. **Transisi:** Sistem menarik panel **Bottom Sheet Detail Aset** dari bawah layar, menampilkan riwayat harga beli rata-rata.

4. **Aksi - Tambah Transaksi:** Di dalam *Bottom Sheet*, pengguna menekan tombol "Catat Jual/Beli" -> Sistem membuka **Form Transaksi Investasi** (Layar Sekunder).
5. **Hasil:** Pengguna mengisi nominal dan mengeksekusi -> Sistem menutup form, mengirim *payload* untuk dienkripsi Rust, dan memuat ulang data di Layar Portofolio.

- **Alur Interaksi Asisten Finansial (AI Nudges)** Alur ini menangani respons pengguna terhadap analitik perilaku.
  1. **Layar Notifikasi:** Terdapat indikator titik merah (jumlah pesan belum dibaca) pada ikon lonceng di navigasi bawah. Pengguna mengetuknya.
  2. **Aksi - Tinjau Saran:** Pengguna membaca kartu saran finansial (misal: peringatan sisa anggaran).
  3. **Aksi - Respon:** * *Opsi 1:* Pengguna menggeser kartu ke kiri (*swipe left*) -> Pesan diabaikan/dihapus dari antarmuka.
  - *Opsi 2:* Pengguna menekan tombol "Tandai Dibaca" -> Sistem mengirim *request* PATCH ke *backend* -> Kartu berubah visual menjadi abu-abu redup (status selesai).
- **Alur Perlindungan Privasi Global (Toggle Sensor)** Fungsionalitas yang dapat diakses dari Layar Beranda maupun Portofolio.
  1. **Aksi - Aktifkan Sensor:** Pengguna mengetuk ikon "Mata" (*Eye Toggle*) di dekat total kekayaan bersih.
  2. **Hasil Instan:** Seluruh elemen teks dalam aplikasi yang memiliki *class/state* `currency_value` (baik itu saldo dompet, jumlah pengeluaran, maupun harga aset) langsung berubah menjadi karakter asteris (Rp `***.***`). Mengetuknya kembali akan memulihkan tampilan angka.

4. Desain Antarmuka Komunikasi

Sistem menggunakan arsitektur layanan mikro berbasis RESTful API (*stateless*). Format pertukaran data standar untuk muatan permintaan (*request*) dan respons adalah JSON. Untuk fase MVP, autentikasi klien diwajibkan menggunakan *JSON Web Tokens* (JWT) pada *header* permintaan, sementara nilai finansial ditransmisikan dalam format teks biasa (*plain text*).

4.1. Modul Autentikasi (Klien ↔ Rust Backend)

A. Registrasi Pengguna Baru

- **URI & Method:** POST /api/v1/auth/register
- **Autentikasi:** Tidak (Public)

Parameter Permintaan:

Nama Parameter	Tipe Data	Wajib	Deskripsi
username	String	Ya	Nama Pengguna yang akan tampil di aplikasi.
email	String	Ya	Alamat surel pengguna yang valid dan unik.
password	String	Ya	Kata sandi dengan panjang minimum 8 karakter.

Parameter Balasan:

Nama Parameter	Tipe Data	Deskripsi
status	String	Indikator keberhasilan (Contoh: "success").
data.user	JSON	Objek data pengguna yang baru saja mendaftar.

B. Login & Penerbitan Token

- **URI & Method:** POST /api/v1/auth/login
- **Autentikasi:** Tidak (Public)

Parameter Permintaan:

Nama Parameter	Tipe Data	Wajib	Deskripsi
email	String	Ya	Alamat surel pengguna yang terdaftar.
password	String	Ya	Kata sandi akun.

Parameter Balasan:

Nama Parameter	Tipe Data	Deskripsi
status	String	Indikator keberhasilan (Contoh: "success").
data.token	String	Token JWT otorisasi klaim untuk sesi klien.
data.expires_in	Integer	Masa berlaku token dalam satuan detik (contoh: 86400).

C. Ambil Profil Pengguna Saat Ini

URI & Method: `GET /api/v1/users/me`

Autentikasi: Ya (JWT)

Parameter Balasan: `data.username`, `data.email`, `data.joined_at`.

4.2. Modul Manajemen Dompet Kas (Klien ↔ Rust Backend)

A. Tambah Dompet Kas Baru

- **URI & Method:** POST /api/v1/wallets
- **Autentikasi:** Ya (JWT)

Parameter Permintaan:

Nama Parameter	Tipe Data	Wajib	Deskripsi
name	String	Ya	Nama referensi dompet (Contoh: "BCA Utama").
initial_balance	Decimal	Ya	Saldo awal pembukaan dompet.

B. Ambil Daftar Dompet Pengguna

- **URI & Method:** GET /api/v1/wallets
- **Autentikasi:** Ya (JWT)
- **Parameter Permintaan:** Tidak ada.

Parameter Balasan:

Nama Parameter	Tipe Data	Deskripsi
data.wallets	Array	Daftar dompet kas yang dimiliki pengguna.
wallets[].wallet_id	Integer	ID unik dompet.
wallets[].name	String	Nama dompet (Contoh: "BCA Utama").
wallets[].balance	Decimal	Saldo kas saat ini di dompet tersebut.

4.3. Modul Pencatatan & Histori Transaksi (Klien ↔ Rust Backend)

A. Pencatatan Transaksi Cerdas via AI

- **URI & Method:** POST /api/v1/transactions
- **Autentikasi:** Ya (JWT)

Parameter Permintaan:

Nama Parameter	Tipe Data	Wajib	Deskripsi
wallet_id	Integer	Ya	ID unik dari dompet sumber pemotongan dana.
raw_text	String	Ya	Teks alami input pengguna (Contoh: "Bayar WiFi 300rb").
amount	Decimal	Ya	Nominal transaksi (Tidak boleh bernilai negatif).

Parameter Balasan:

Nama Parameter	Tipe Data	Deskripsi
status	String	Indikator keberhasilan.
data.transaction_id	String	UUID transaksi di dalam basis data.
data.category_name	String	Label pengeluaran hasil klasifikasi layanan AI.
data.ai_confidence	Float	Nilai metrik keyakinan AI (0.0 hingga 1.0).
data.needs_validation	Boolean	Bernilai true jika AI kurang yakin (< 0.70).

B. Ambil Riwayat Transaksi

- **URI & Method:** GET /api/v1/transactions
- **Autentikasi:** Ya (JWT)

Parameter Permintaan (Query String):

Nama Parameter	Tipe Data	Wajib	Deskripsi
limit	Integer	Tidak	Batas jumlah baris yang diretur (Default: 20).
offset	Integer	Tidak	Titik awal data untuk paginasi (Default: 0).

Parameter Balasan:

Nama Parameter	Tipe Data	Deskripsi

data.transactions	Array	Daftar transaksi harian pengguna.
transactions[].raw_text	String	Teks asli transaksi.
transactions[].category_name	String	Kategori transaksi saat ini.

C. Koreksi Kategori Transaksi

- **URI & Method:** PUT /api/v1/transactions/{transaction_id}
- **Autentikasi:** Ya (JWT)

Parameter Permintaan:

Nama Parameter	Tipe Data	Wajib	Deskripsi
category_id	Integer	Ya	ID kategori baru yang dipilih secara manual oleh pengguna.

4.5. Modul Agregasi Portofolio (Klien ↔ Rust Backend)

A. Catat Mutasi Jual/Beli Aset (Investasi)

- **URI & Method:** POST /api/v1/investments
- **Autentikasi:** Ya (JWT)

Parameter Permintaan:

Nama Parameter	Tipe Data	Wajib	Deskripsi
asset_ticker	String	Ya	Simbol instrumen (Contoh: "BBCA.JK", "ETH").
transaction_type	String	Ya	Jenis mutasi: "buy" atau "sell".

quantity	Decimal	Ya	Jumlah lot/koin yang ditransaksikan.
execution_price	Decimal	Ya	Harga eksekusi aktual saat transaksi terjadi.

B. Muat Dasbor Portofolio (PnL Real-Time)

- **URI & Method:** GET /api/v1/portfolio/summary
- **Autentikasi:** Ya (JWT)
- **Parameter Permintaan:** Tidak ada.

Parameter Balasan:

Nama Parameter	**Tipe Data**	**Deskripsi**
data.total_net_worth	Decimal	Total kekayaan bersih agregat kas dan investasi (IDR).
data.assets	Array	Kumpulan rincian aset individu.
assets[].symbol	String	Ticker instrumen publik.
assets[].quantity	Decimal	Jumlah kepemilikan lot atau keping koin.
assets[].current_price	Decimal	Harga satuan aktual dari layanan eksternal.
assets[].unrealized_pnl	Decimal	Nominal selisih keuntungan/kerugian (IDR).
assets[].pnl_percentage	Float	Persentase rasio keuntungan/kerugian.

4.6. Modul Asisten Finansial (Klien ↔ Rust Backend)

A. Ambil Daftar Saran Belum Dibaca

- **URI & Method:** GET /api/v1/insights?status=unread
- **Autentikasi:** Ya (JWT)
- **Parameter Permintaan:** Tidak ada.

Parameter Balasan:

Nama Parameter	Tipe Data	Deskripsi
data.insights	Array	Daftar pesan saran.
insights[].insight_id	Integer	ID unik pesan dari tabel Insights.
insights[].type	String	Klasifikasi pesan (Contoh: "budget_alert").
insights[].message	String	Teks narasi saran finansial proaktif.

B. Tandai Saran Telah Dibaca

- **URI & Method:** PATCH /api/v1/insights/{insight_id}/read
- **Autentikasi:** Ya (JWT)
- **Parameter Permintaan:** Tidak ada *payload* tambahan.

4.7. Standar Kode Kesalahan Global (Error Codes)

Kode HTTP	Deskripsi Fungsional & Mitigasi
400 Bad Request	Tipe data parameter tidak valid atau aturan input dilanggar (misal harga negatif).

401 Unauthorized	Token JWT kedaluwarsa atau tidak ada. Antarmuka wajib memuat ulang Layar Login.
403 Forbidden	Pengguna mencoba memanipulasi entitas (wallet_id, transaction_id) milik pengguna lain.
404 Not Found	Data referensi (misal Ticker Simbol) tidak dikenali oleh sistem.
429 Too Many Req.	Pembatasan laju (*Rate-Limiting*) tercapai untuk mencegah beban pada layanan eksternal.
500 Internal Error	Kegagalan peladen kritis (koneksi MySQL terputus atau layanan Python tumbang).

5. Desain Komponen dan Logika Bisnis

Bab ini mendeskripsikan algoritma internal, aliran kontrol, dan pemrosesan data di dalam arsitektur layanan mikro (*microservices*). Untuk fase *Minimum Viable Product* (MVP), logika bisnis difokuskan pada kecepatan komputasi agregasi aset dan orkestrasi inferensi *Machine Learning*.

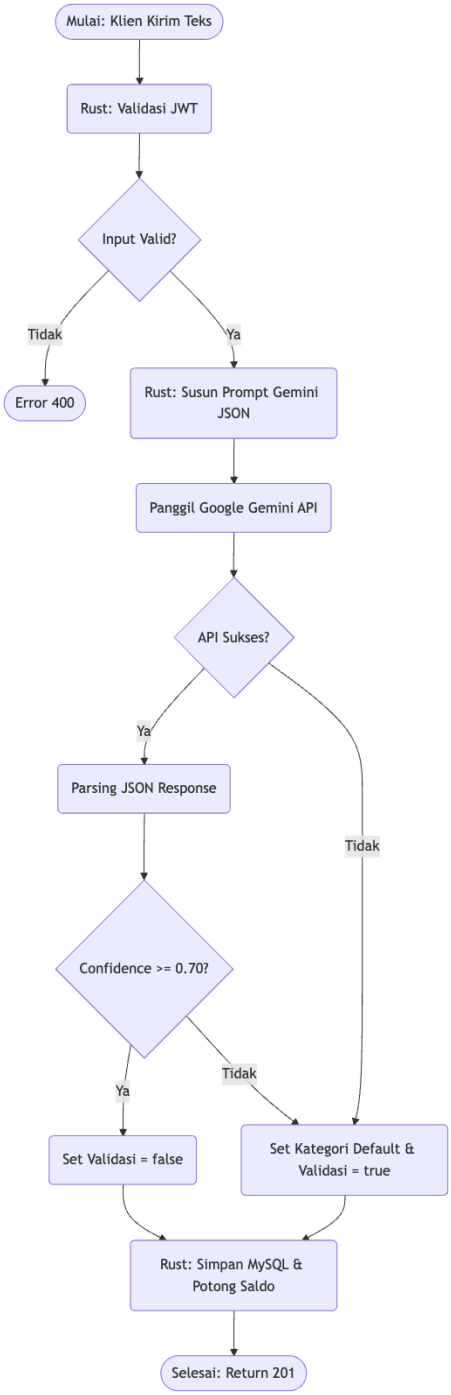
5.1. Alur Pemrosesan Pencatatan Transaksi (Rust ↔ Gemini API)

Komponen ini mengatur siklus hidup data transaksi pengeluaran sejak diinput oleh pengguna hingga disimpan secara permanen.

Algoritma Eksekusi:

1. **Penerimaan & Validasi:** Peladen Rust menerima *payload* JSON dari klien. Sistem memvalidasi token JWT untuk mengekstrak user_id dan memastikan amount bernilai positif.
2. **Delegasi Kognitif (Asinkron):** Rust menangguhkan (*suspend*) proses penyimpanan ke MySQL dan mengirimkan raw_text (contoh: "Bayar WiFi 300rb") ke *endpoint* internal Layanan Gemini.

3. **Evaluasi Aturan Bisnis:** Rust menerima respons AI. Jika skor keyakinan < 0.70 (70%), Rust akan mengubah *flag* needs_validation menjadi true.
4. **Persistensi Data:** Rust memotong saldo dompet pengguna di tabel Wallets (menggunakan *Database Transaction* agar konsisten) dan menyisipkan baris baru ke tabel Transactions beserta ID kategori dari AI.



5.3. Mesin Agregasi Dasbor Portofolio (Rust Core)

Komponen paling krusial yang menyajikan gambaran *Net Worth* pengguna secara *real-time*. Komputasi ini memanfaatkan fitur asinkron Rust (tokio) untuk mencegah *blocking* saat memanggil harga pasar.

1. **Pengambilan Posisi Aset:** Rust menjalankan kueri ke MySQL untuk mengambil daftar aset investasi pengguna beserta total Lot/Kuantitas dan Harga Beli Rata-rata
2. **Pengambilan Harga Pasar Paralel:** Untuk setiap simbol aset (misal: BBCA.JK, ETH), Rust memicu fungsi penarik data (*fetcher*) dari pustaka seperti yfinance-rs secara serentak (konkurensi). Jika ada 5 aset, 5 panggilan jaringan dilakukan bersamaan untuk mendapatkan Harga Saat Ini.
3. **Komputasi Matematis (In-Memory):**
   - **Total Nilai Aktual (Total Value):**

     $$Total_Value = P_{current} \times q$$

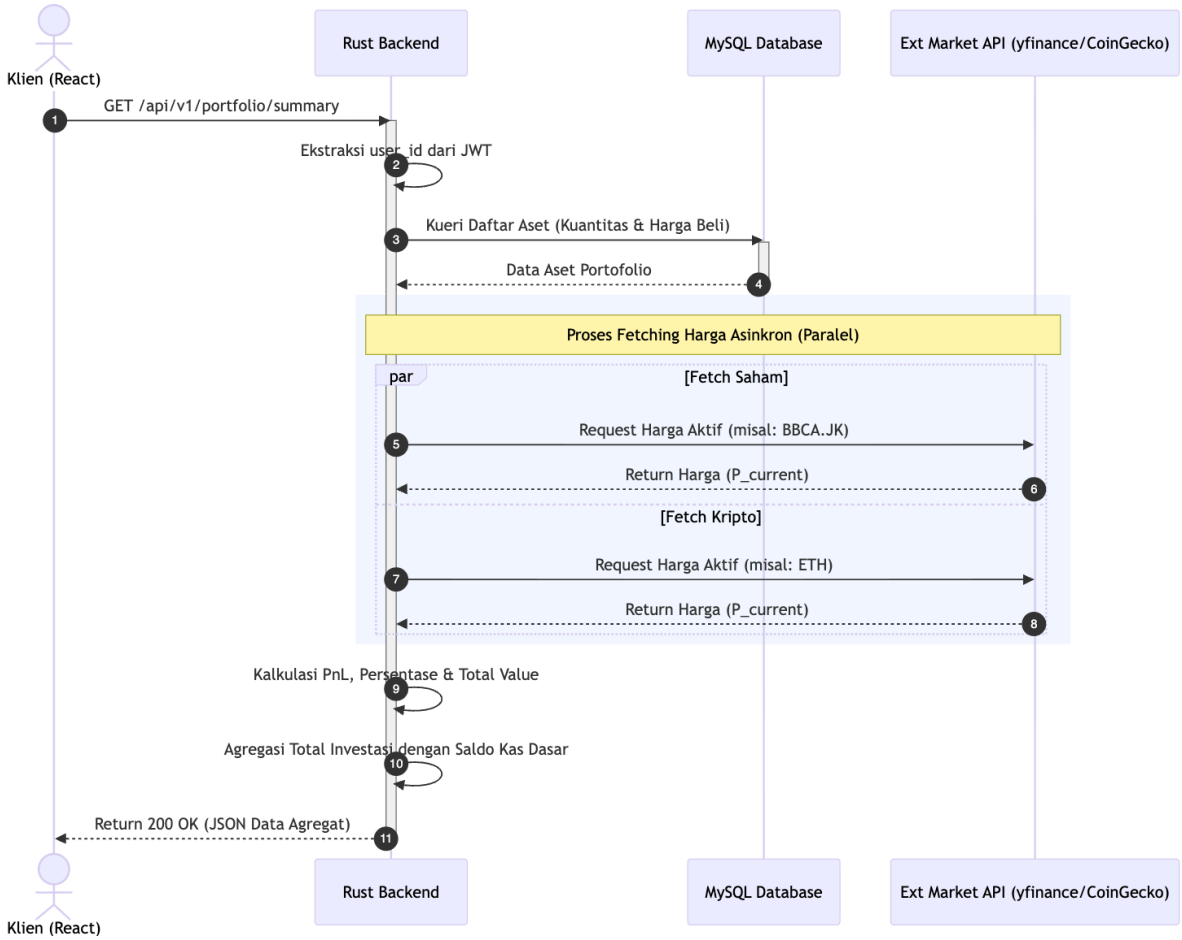
   - **Keuntungan/Kerugian Belum Terealisasi (Unrealized PnL):**

     $$Unrealized_PnL = (P_{current} - P_{avg-buy}) \times q$$

   - **Persentase PnL:**

     $$PnL\,\% = \left(\frac{P_{current} - p_{avg\ buy}}{p_{avg\ buy}}\right) \times 100$$

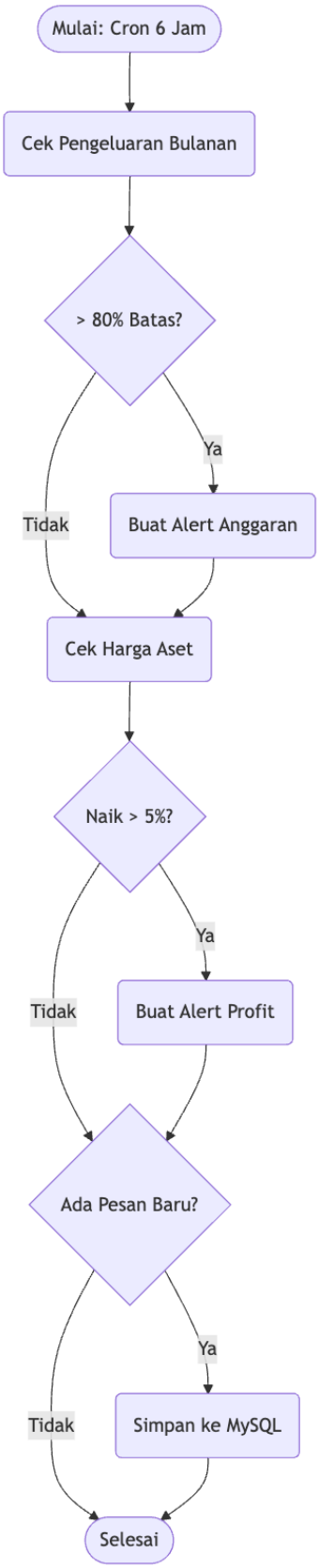
4. **Agregasi Global:** Total nilai aktual dari seluruh investasi ditambahkan dengan total saldo kas cair dari tabel Wallets untuk menghasilkan parameter akhir total_net_worth.



5.4. Generator Asisten Finansial (Background Job)

Komponen proaktif yang berjalan di latar belakang (tanpa interaksi langsung dari pengguna) untuk mendeteksi anomali finansial.

1. **Pemicu:** Sistem penjadwalan berjalan setiap interval tertentu (misalnya, setiap 6 jam sekali).
2. **Pemeriksaan Anggaran (Budget Checking):** Sistem membandingkan total pengeluaran pengguna di bulan berjalan dengan rata-rata historis bulan sebelumnya. Jika pengeluaran >80% sebelum pertengahan bulan, sistem membuat rekaman peringatan baru di tabel Insights.
3. **Pemeriksaan Portofolio:** Sistem memantau PnL. Jika sebuah aset kripto mengalami kenaikan tiba-tiba lebih dari 5%dalam sehari, sistem merakit pesan saran "*Profit taking*".
4. **Penyampaian:** Pesan disimpan dengan status *unread*. Saat klien memanggil GET /api/v1/insights, pesan ini ditarik untuk ditampilkan sebagai notifikasi di antarmuka pengguna.


Mulai: Cron 6 Jam
Cek Pengeluaran Bulanan
> 80% Batas?
Ya
Tidak
Buat Alert Anggaran
Cek Harga Aset
Naik > 5%?
Ya
Tidak
Buat Alert Profit
Ada Pesan Baru?
Ya
Tidak
Simpan ke MySQL
Selesai

7. Keamanan dan Penanganan Kesalahan

Bab ini menjabarkan mekanisme pertahanan sistem terhadap ancaman keamanan siber serta strategi resiliensi perangkat lunak dalam menangani anomali teknis pada fase *Minimum Viable Product* (MVP).

7.1. Manajemen Identitas dan Sesi (Authentication)

- **Hashing Kata Sandi:** Kata sandi pengguna tidak pernah disimpan dalam bentuk teks biasa (*plain text*). Peladen Rust menggunakan algoritma *hashing* kriptografik modern (seperti Argon2 atau Bcrypt) dengan *salt* acak sebelum menyimpannya ke tabel `Users` di MySQL.
- **Otorisasi Berbasis Token (JWT):** Sesi pengguna dikelola menggunakan *JSON Web Tokens* (JWT) yang bersifat *stateless*. Token ditandatangani menggunakan *Secret Key* di peladen dengan algoritma HMAC SHA-256 (HS256). Token memiliki masa berlaku terbatas (contoh: 24 jam) untuk memitigasi risiko pembajakan sesi.
- **Isolasi Penyewa (*Tenant Isolation*):** Setiap *endpoint* API yang memanipulasi data wajib memvalidasi `user_id` yang diekstrak dari JWT. Kueri basis data selalu menyertakan klausa `WHERE user_id = ?` untuk mencegah eksploitasi ID (BOLA/IDOR), sehingga pengguna tidak dapat melihat atau mengubah dompet/transaksi milik pengguna lain.

7.2. Keamanan Jaringan dan Layanan Pihak Ketiga

- **Enkripsi Transportasi (TLS/HTTPS):** Seluruh lalu lintas komunikasi antara antarmuka React klien dan gerbang API peladen Rust wajib berjalan di atas protokol HTTPS (TLS 1.2/1.3) untuk mencegah penyadapan data (*Man-in-the-Middle Attack*).
- **Manajemen Kunci Kredensial (API Keys):** Kunci rahasia untuk mengakses Google Gemini API dan penyedia data harga pasar tidak pernah ditulis langsung di dalam kode sumber (*hardcoded*). Kunci tersebut disimpan dan dikelola secara aman menggunakan variabel lingkungan (*Environment Variables*) atau layanan terkelola seperti **AWS Secrets Manager**.
- **Pembatasan Laju (Rate Limiting):** Untuk mencegah serangan *DDoS* dan memitigasi risiko pembengkakan tagihan API eksternal (*billing exhaustion*), *endpoint* publik dibatasi laju panggilannya berdasarkan alamat IP klien.

7.3. Keamanan Basis Data dan Sanitasi Input

- **Mitigasi Injeksi SQL:** Sistem tidak pernah merakit kueri SQL menggunakan penyambungan untai (*string concatenation*). Peladen Rust menggunakan *Prepared Statements* (melalui pustaka ORM) untuk memisahkan parameter data dari perintah SQL.
- **Sanitasi Input XSS:** Parameter teks masukan pengguna, khususnya `raw_text` pada input transaksi, divalidasi dan disanitasi dari injeksi *script* berbahaya sebelum diteruskan ke Gemini API atau disimpan ke basis data.

7.4. Strategi Penanganan Kesalahan (Error Handling & Fallback)

Sistem dirancang dengan prinsip degradasi anggun (*graceful degradation*) agar tetap beroperasi secara parsial ketika layanan eksternal mengalami gangguan.

- **Format Kesalahan Terstandarisasi:** Seluruh *endpoint* mengembalikan balasan kesalahan dengan struktur JSON yang konsisten, memuat kode status HTTP semantik beserta pesan yang dapat dibaca manusia agar UI React dapat merender instruksi yang jelas.
- **Sistem Cadangan AI (*AI Fallback*):** Jika layanan Google Gemini API mengalami gangguan koneksi (*timeout*), membalas dengan status galat (5xx), atau melampaui batas kuota pemanggilan (*Rate Limit* 429), peladen Rust **tidak akan membatalkan** proses penyimpanan transaksi pengguna. Alih-alih merespons dengan galat (yang akan merusak pengalaman pengguna), Rust secara otomatis akan menyimpan transaksi tersebut dengan kategori *default* (misalnya: "Lainnya") dan mengatur bendera `needs_validation = true`, sehingga pengguna tetap bisa mencatat arus kasnya tanpa hambatan operasional.
- **Pencatatan Terstruktur (*Structured Logging*):** Peladen Rust diinstrumentasi untuk mencatat jejak eksekusi setiap *request*, tingkat keparahan (*INFO, WARN, ERROR*), dan respons galat dari API pihak ketiga ke dalam sistem monitor log untuk mempercepat proses pengawakutuan (*debugging*).

8. Infrastruktur dan Rilis

Bab ini mendefinisikan arsitektur topologi jaringan, lingkungan peladen (*server environment*), dan alur pengiriman terintegrasi (*CI/CD*) yang digunakan untuk mendistribusikan aplikasi secara aman pada fase MVP menggunakan ekosistem **Amazon Web Services (AWS)**.

8.1. Topologi Lingkungan Peladen (AWS Environment)

Sistem ini menggunakan pendekatan *containerization* dan layanan terkelola (*managed services*) dari AWS untuk menekan beban pemeliharaan infrastruktur secara drastis, mengingat sistem kini murni beroperasi dengan satu peladen *backend*.

- **Frontend (Aplikasi Klien React):** Di-hosting menggunakan **AWS Amplify** (atau Amazon S3 + CloudFront). Pendekatan ini memastikan aset statis antarmuka pengguna didistribusikan melalui *Content Delivery Network* (CDN) global AWS dengan latensi yang sangat rendah.
- **Backend Core (Rust API Gateway):** Dibungkus ke dalam sebuah *Docker Image*. Untuk fase MVP, kontainer peladen Rust ini dijalankan di dalam instans **Amazon EC2 (Elastic Compute Cloud)**. Rust yang terkenal sangat hemat memori (RAM) memungkinkan aplikasi ini berjalan dengan sangat lancar bahkan pada instans EC2 berukuran kecil (seperti *t3.micro* atau *t3.small*), menjadikannya sangat hemat biaya.
- **Basis Data (MySQL):** Menggunakan layanan **Amazon RDS (Relational Database Service) for MySQL**. RDS menyediakan basis data relasional yang terkelola penuh, termasuk fitur pencadangan otomatis (*automated backups*) dan pembaruan keamanan, sehingga data finansial pengguna jauh lebih aman dibandingkan mengelolanya secara manual.

8.2. Arsitektur Jaringan dan Keamanan (AWS VPC)

Infrastruktur jaringan diisolasi secara logis menggunakan **Amazon VPC (Virtual Private Cloud)** untuk membatasi akses dari internet publik.

- **Isolasi Subnet & Security Groups:**
  - Instans Amazon EC2 (peladen Rust) diletakkan di dalam *Public Subnet* dan dilindungi oleh *AWS Security Groups* yang hanya mengizinkan lalu lintas masuk (*inbound traffic*) pada port HTTP (80) dan HTTPS (443).
  - Instans Amazon RDS diletakkan di dalam *Private Subnet*. RDS tidak memiliki akses internet publik dan hanya dapat diakses secara eksklusif oleh instans EC2 yang berada di dalam VPC yang sama.
- **Reverse Proxy:** Layanan Nginx dikonfigurasi di dalam instans EC2 untuk bertindak sebagai *Reverse Proxy*, melakukan terminasi SSL/TLS (menggunakan sertifikat Let's Encrypt), dan meneruskan (*routing*) seluruh rute `/api/v1/*` langsung ke port internal peladen Rust.

8.3. Alur Integrasi dan Pengiriman Berkelanjutan (CI/CD Pipeline)

Untuk meminimalisasi kesalahan manusia saat rilis pembaruan kode, sistem mengimplementasikan jalur pipa CI/CD otomatis menggunakan **GitHub Actions** yang terhubung langsung dengan AWS.

Alur Kerja Rilis (Deployment Workflow):

1. **Push & Trigger:** Pengembang melakukan *push* kode ke cabang `main` di repositori GitHub.
2. **Build & Containerize:** GitHub Actions memicu tugas (*runner*) untuk melakukan kompilasi peladen Rust (mode `--release`) dan merakit ulang *Docker Image*.
3. **Container Registry:** Image Docker Rust yang telah dirakit didorong (*push*) ke dalam **Amazon ECR (Elastic Container Registry)** yang bertindak sebagai repositori artefak privat milik sistem.
4. **Automated Deployment:** GitHub Actions mengirimkan perintah peluncuran (via SSH atau AWS Systems Manager) ke instans EC2 untuk menarik (*pull*) image Rust terbaru dari ECR dan menjalankan ulang kontainer tanpa waktu henti (*zero-downtime deployment*).

8.4. Pemantauan dan Log (Monitoring & Observability)

- **Log Terpusat:** Sistem memanfaatkan **Amazon CloudWatch** untuk memusatkan pencatatan log (*centralized logging*). Metrik penggunaan CPU/RAM instans EC2, log aplikasi Rust, serta metrik basis data RDS diekspor ke CloudWatch untuk analisis aktivitas.
- **Notifikasi Anomali:** Konfigurasi *CloudWatch Alarms* dipasang untuk memantau status kesehatan gerbang API (`/api/v1/health`) dan lonjakan galat dari eksekusi Gemini API. Jika terdeteksi waktu henti (*downtime*), sistem akan memicu layanan **Amazon SNS** untuk segera mengirimkan surel peringatan ke tim pengembang.